॥ वसुधैव कुटुम्बकम् ॥

# SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

**(Established under section 3 of the UGC Act 1956)**

**Re-accredited by NAAC with "A" Grade**

**Name of the Student: Vatsal Postiwala** **PRN: 26070122509**

**Semester: III** **Year: 2026-27**

**Subject Title: Operating Systems Lab**

**EXPERIMENT No**: 10

**TITLE**: Priority Scheduling Algorithm **DoP**: 28/09/2026

**Aim:** Implement a C++ program to demonstrate the Priority Scheduling algorithm.

**Learning Outcome:** 1. To understand the scheduling algorithm
2. To Demonstrate the Priority algorithm

**Hardware/Software:**

- Hardware requirements:
  - A PC with a minimum of 4GB RAM.
  - A processor with 1.6GHz clock speed or faster.
  - Sufficient hard disk space to store the code and compiler.
- Software Requirements:
  - Visual Studio Code (VS Code)
  - C compiler such as GCC
  - C/C++ extension for VS Code. This extension is used to compile and run C++ code directly from VS Code.

**Theory:**

Scheduling algorithms are used in operating systems to allocate system resources to processes in an efficient manner. Scheduling algorithms determine which process gets access to the CPU and when, based on predefined criteria such as priority, execution time, and resource requirements.

Priority scheduling is a scheduling algorithm where each process is assigned a priority, and the process with the highest priority is executed first. This scheduling method can be preemptive or non-preemptive. In preemptive priority scheduling, the process with the highest priority can be interrupted by a process with an even higher priority. This method ensures that high-priority processes are executed first, reducing waiting times for important tasks. However, it can lead to starvation of lower-priority tasks. In non-preemptive priority scheduling, the CPU is allocated to a process with the highest priority, and the process is executed until it completes or goes into a waiting state. This method can also lead to starvation of lower-priority tasks, but it is less likely to

happen than in preemptive priority scheduling

The features of the Priority algorithm include:

- Prioritization: Each process is assigned a priority level, which determines its order of execution. The highest-priority process is executed first.
- Maybe Preemptive or non-preemptive
  Starvation prevention: Priority scheduling can prevent starvation by ensuring that lower-priority processes are eventually executed. This can be achieved by using aging techniques or by periodically boosting the priority of lower-priority processes.
- Multiple queues: Priority scheduling can be implemented using multiple queues, each with its own priority level. This can improve the efficiency of the scheduler by reducing the search time for the highest-priority process.
- Multiple queues: Priority scheduling can be implemented using multiple queues, each with its own priority level. This can improve the efficiency of the scheduler by reducing the search time for the highest-priority process.

The drawbacks of the Priority algorithm include:

- Starvation: If a process with a lower priority never gets a chance to execute, it may suffer from starvation
- Indefinite blocking: If a process with a high priority is waiting for a resource that is currently being used by a lower-priority process, it may be indefinitely blocked.
  Priority inversion: Lower-priority processes can prevent higher-priority processes from executing in certain situations. This can happen if a higher-priority process is waiting for a resource that is being held by a lower-priority process.
- Priority scheduling can be more complex than other scheduling algorithms, such as FCFS or Round Robin, which can make it more difficult to implement and debug

**Problem Statement:**

- **Calculate following using preemptive Priority Scheduling**

  - Gantt Chart
- Completion Time (CT) for every process
- Turnaround Time (TAT)
- Waiting Time (WT)
- Response Time (RT)
- Average Waiting Time
- Average Turnaround Time
- Number of context switches
- Number of preemptions for each process
- Identify the process with maximum WT and maximum RT.

| Process | Arrival Time (AT) | Burst Time (BT) | Priority |
|---|---|---|---|
| P1 | 0 | 14 | 4 |
| P2 | 1 | 6 | 2 |
| P3 | 3 | 8 | 5 |
| P4 | 5 | 4 | 1 |
| P5 | 6 | 7 | 3 |
| P6 | 8 | 5 | 2 |
| P7 | 11 | 3 | 1 |
| P8 | 13 | 6 | 3 |

| Process | Arrival Time (AT) | Burst Time (BT) | Priority |
|---|---|---|---|
| P1 | 0 | 10 | 3 |
| P2 | 1 | 4 | 1 |
| P3 | 2 | 7 | 4 |
| P4 | 4 | 5 | 2 |
| P5 | 6 | 3 | 1 |
| P6 | 7 | 6 | 3 |
| P7 | 9 | 4 | 2 |
| P8 | 11 | 5 | 1 |
| P9 | 13 | 3 | 4 |
| P10 | 15 | 4 | 2 |

**Algorithm:**

PAGE NO.
DATE:

★ Algorithm:-

1) Start
2) Read the No. of Processes
3) For each Process, enter:
   - Process ID
   - AT
   - BT
   - Priority
4) set the remaining burst time of every process equal to its burst time.
5) Set the current time to 0 & Completed process to 0.
6) Find all process where arrival time is less than or equal to the current time & where remaining burst time is greater than 0.
7) Select the process having the highest priority. If priority are equal, select the process that arrived earlier.
8) If no process is available, increase the time by 1.
9) If the higher priority process arrives while another process is running prompt the current process.
10) Record the first execution time of every process & Calculate its response time.
11) Execute the related process for one unit of CPU time & decreases its remaining burst time by 1.
12) Increase the current time by 1.

PAGE NO.
DATE:

12) If the remaining burst time becomes 0, calculate:-
   → CT
   → TAT
   → WT

14) Repeat steps 6-13 until all processes are completed.

15) Calculate:- AWT, ATAT, No of context switches, No. of preemptions.

16) Find the processes have maximum waiting time & maximum Response Time.

17) Display the gantt chart & all calculated values

18) Stop.

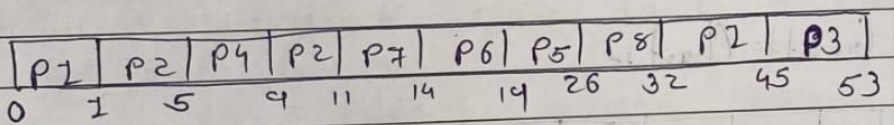
# Pre-emptive Priority Schedul

PAGE NO.
DATE

1]

| Process | AT | BT | Priority | CT | TAT | WT |
|---|---|---|---|---|---|---|
| P1 | 0 | 14 | 4 | 45 | 45 | 31 |
| P2 | 1 | 6 | 2 | 11 | 10 | 4 |
| P3 | 3 | 8 | 5 | 53 | 50 | 42 |
| P4 | 5 | 4 | 1 | 9 | 4 | 0 |
| P5 | 6 | 7 | 3 | 26 | 20 | 13 |
| P6 | 8 | 5 | 2 | 19 | 11 | 6 |
| P7 | 11 | 3 | 1 | 14 | 3 | 0 |
| P8 | 13 | 6 | 3 | 32 | 19 | 13 |

Gantt Chart:-




Avg WT = 104/8 = 13.625 ms

Avg TAT = 162/8 = 20.25 ms

PAGE NO.
DATE:

2)

| Process | AT | BT | Priority | CT | TAT | WT |
|---|---|---|---|---|---|---|
| P1 | 0 | 10 | 3 | 35 | 35 | 25 |
| P2 | 1 | 4 | 1 | 5 | 4 | 0 |
| P3 | 2 | 7 | 4 | 48 | 46 | 39 |
| P4 | 4 | 5 | 2 | 18 | 14 | 9 |
| P5 | 6 | 3 | 1 | 9 | 3 | 0 |
| P6 | 7 | 6 | 3 | 41 | 34 | 28 |
| P7 | 9 | 4 | 2 | 22 | 13 | 9 |
| P8 | 11 | 5 | 1 | 16 | 5 | 0 |
| P9 | 13 | 3 | 4 | 51 | 38 | 35 |
| P10 | 15 | 4 | 2 | 26 | 11 | 7 |

Gantt Chart :-

| P1 | P2 | P4 | P5 | P4 | P8 | P4 | P7 | P10 | P2 | P6 | P3 | P9 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 5 | 6 | 9 | 11 | 16 | 18 | 22 | 26 | 35 | 41 | 48 | 51 |

Avg WT = 152 / 10 = 15.2

Avg TAT = 203 / 10 = 20.3

**Program:**
**Code :- 1 .**

```
#include <stdio.h>

struct Process
{
   int pid;
   int at;
   int bt;
   int priority;
   int remaining;
   int ct;
   int tat;
   int wt;
   int rt;
   int first_start;
   int preemptions;
};

int main()
{
   struct Process p[8] =
   {
      {1, 0, 14, 4, 14, 0, 0, 0, -1, -1, 0},
      {2, 1,  6, 2,  6, 0, 0, 0, -1, -1, 0},
      {3, 3,  8, 5, 8, 0, 0, 0, -1, -1, 0},
      {4, 5,  4, 1, 4, 0, 0, 0, -1, -1, 0},
      {5, 6,  7, 3, 7, 0, 0, 0, -1, -1, 0},
      {6, 8,  5, 2, 5, 0, 0, 0, -1, -1, 0},
      {7, 11, 3, 1, 3, 0, 0, 0, -1, -1, 0},
      {8, 13, 6, 3, 6, 0, 0, 0, -1, -1, 0}
   };

   int n = 8;
   int time = 0;
   int completed = 0;
   int current = -1;
   int context_switches = 0;

   float total_wt = 0;
   float total_tat = 0;

   printf("\nPREEMPTIVE PRIORITY SCHEDULING - QUESTION 1\n");

   printf("\nGANTT CHART:\n");

   while (completed < n)
   {
      int selected = -1;

      /* Find highest priority process */
```

```
for (int i = 0; i < n; i++)
{
   if (p[i].at <= time && p[i].remaining > 0)
   {
      if (selected == -1 ||
         p[i].priority < p[selected].priority)
      {
         selected = i;
      }
   }
}

/* CPU idle */
if (selected == -1)
{
   time++;
   continue;
}

/* Process changed */
if (current != selected)
{
   if (current != -1)
   {
      context_switches++;

      if (p[current].remaining > 0)
      {
         p[current].preemptions++;
      }
   }

   current = selected;

   /* First CPU execution */
   if (p[current].first_start == -1)
   {
      p[current].first_start = time;
      p[current].rt =
         time - p[current].at;
   }

   printf("| P%d ", p[current].pid);
}

/* Execute for one unit */
p[current].remaining--;
time++;

/* Process completed */
if (p[current].remaining == 0)
```

```
    {
        p[current].ct = time;

        p[current].tat =
            p[current].ct - p[current].at;

        p[current].wt =
            p[current].tat - p[current].bt;

        completed++;
        current = -1;
    }
}

printf("|\n");

/* Display result */
printf("\n----------------------------------------------------------------\n");

printf("PID\tAT\tBT\tPriority\tCT\tTAT\tWT\tRT\tPreemptions\n");

printf("----------------------------------------------------------------\n");

int max_wt = -1;
int max_rt = -1;
int max_wt_pid = 0;
int max_rt_pid = 0;

for (int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\t%d\t%d\n",
           p[i].pid,
           p[i].at,
           p[i].bt,
           p[i].priority,
           p[i].ct,
           p[i].tat,
           p[i].wt,
           p[i].rt,
           p[i].preemptions);

    total_wt += p[i].wt;
    total_tat += p[i].tat;

    if (p[i].wt > max_wt)
    {
        max_wt = p[i].wt;
        max_wt_pid = p[i].pid;
    }

    if (p[i].rt > max_rt)
```

```
        {
            max_rt = p[i].rt;
            max_rt_pid = p[i].pid;
        }
    }

    printf("\nAverage Waiting Time = %.2f",
           total_wt / n);

    printf("\nAverage Turnaround Time = %.2f",
           total_tat / n);

    printf("\nContext Switches = %d",
           context_switches);

    printf("\nMaximum Waiting Time = P%d (%d)",
           max_wt_pid, max_wt);

    printf("\nMaximum Response Time = P%d (%d)",
           max_rt_pid, max_rt);

    printf("\n");

    return 0;
}
```

**Output :-**

```
vatsal@Ubuntu-VirtualBox:~/OS$ nano exp10a.c
vatsal@Ubuntu-VirtualBox:~/OS$ gcc exp10a.c -o exp10a
vatsal@Ubuntu-VirtualBox:~/OS$ ./exp10a

PREEMPTIVE PRIORITY SCHEDULING - QUESTION 1

GANTT CHART:
| P1 | P2 | P4 | P2 | P7 | P6 | P5 | P8 | P1 | P3 |

------------------------------------------------------------
PID     AT      BT      Priority        CT      TAT     WT
------------------------------------------------------------
P1      0       14      4               45      45      31
P2      1       6       2               11      10      4
P3      3       8       5               53      50      42
P4      5       4       1               9       4       0
P5      6       7       3               26      20      13
P6      8       5       2               19      11      6
P7      11      3       1               14      3       0
P8      13      6       3               32      19      13

Average Waiting Time = 13.62
Average Turnaround Time = 20.25
```

**Code :- 2.**

```
#include <stdio.h>

struct Process
{
   int pid;
   int at;
   int bt;
   int priority;
   int remaining;
   int ct;
   int tat;
   int wt;
   int rt;
   int first_start;
   int preemptions;
};

int main()
{
   struct Process p[10] =
   {
      {1,  0, 10, 3, 10, 0, 0, 0, -1, -1, 0},
      {2,  1,  4, 1,  4, 0, 0, 0, -1, -1, 0},
      {3,  2,  7, 4,  7, 0, 0, 0, -1, -1, 0},
      {4,  4,  5, 2,  5, 0, 0, 0, -1, -1, 0},
      {5,  6,  3, 1,  3, 0, 0, 0, -1, -1, 0},
      {6,  7,  6, 3,  6, 0, 0, 0, -1, -1, 0},
      {7,  9,  4, 2,  4, 0, 0, 0, -1, -1, 0},
      {8, 11,  5, 1,  5, 0, 0, 0, -1, -1, 0},
      {9, 13,  3, 4,  3, 0, 0, 0, -1, -1, 0},
      {10,15,  4, 2,  4, 0, 0, 0, -1, -1, 0}
   };

   int n = 10;
   int time = 0;
   int completed = 0;
   int current = -1;
   int context_switches = 0;

   float total_wt = 0;
   float total_tat = 0;

   printf("\nPREEMPTIVE PRIORITY SCHEDULING - QUESTION 2\n");

   printf("\nGANTT CHART:\n");

   while (completed < n)
   {
      int selected = -1;
```

```
/* Find highest priority process */
for (int i = 0; i < n; i++)
{
   if (p[i].at <= time && p[i].remaining > 0)
   {
      if (selected == -1 ||
         p[i].priority < p[selected].priority)
      {
         selected = i;
      }
   }
}

/* CPU idle */
if (selected == -1)
{
   time++;
   continue;
}

/* Process changed */
if (current != selected)
{
   if (current != -1)
   {
      context_switches++;

      if (p[current].remaining > 0)
      {
         p[current].preemptions++;
      }
   }

   current = selected;

   /* First CPU execution */
   if (p[current].first_start == -1)
   {
      p[current].first_start = time;

      p[current].rt =
         time - p[current].at;
   }

   printf("| P%d ", p[current].pid);
}

/* Execute for one unit */
p[current].remaining--;
time++;
```

```
    /* Process completed */
    if (p[current].remaining == 0)
    {
        p[current].ct = time;

        p[current].tat =
            p[current].ct - p[current].at;

        p[current].wt =
            p[current].tat - p[current].bt;

        completed++;
        current = -1;
    }
}

printf("|\n");

/* Display result */
printf("\n------------------------------------------------------------\n");

printf("PID\tAT\tBT\tPriority\tCT\tTAT\tWT\tRT\tPreemptions\n");

printf("------------------------------------------------------------\n");

int max_wt = -1;
int max_rt = -1;
int max_wt_pid = 0;
int max_rt_pid = 0;

for (int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\t%d\t%d\n",
           p[i].pid,
           p[i].at,
           p[i].bt,
           p[i].priority,
           p[i].ct,
           p[i].tat,
           p[i].wt,
           p[i].rt,
           p[i].preemptions);

    total_wt += p[i].wt;
    total_tat += p[i].tat;

    if (p[i].wt > max_wt)
    {
        max_wt = p[i].wt;
        max_wt_pid = p[i].pid;
```

```
        }

        if (p[i].rt > max_rt)
        {
            max_rt = p[i].rt;
            max_rt_pid = p[i].pid;
        }
    }

    printf("\nAverage Waiting Time = %.2f",
          total_wt / n);

    printf("\nAverage Turnaround Time = %.2f",
          total_tat / n);

    printf("\nContext Switches = %d",
          context_switches);

    printf("\nMaximum Waiting Time = P%d (%d)",
          max_wt_pid, max_wt);

    printf("\nMaximum Response Time = P%d (%d)",
          max_rt_pid, max_rt);

    printf("\n");

    return 0;
}
```

**Output :-**

```
vatsal@Ubuntu-VirtualBox:~/OS$ nano exp10b.c
vatsal@Ubuntu-VirtualBox:~/OS$ gcc exp10b.c -o exp10b
vatsal@Ubuntu-VirtualBox:~/OS$ ./exp10b

PREEMPTIVE PRIORITY SCHEDULING - QUESTION 2

GANTT CHART:
| P1 | P2 | P4 | P5 | P4 | P8 | P4 | P7 | P10 | P1 | P6 | P3 | P9 |

-------------------------------------------------------------------------
PID     AT      BT      Priority        CT      TAT     WT      RT
-------------------------------------------------------------------------
P1      0       10      3               35      35      25      0
P2      1       4       1               5       4       0       0
P3      2       7       4               48      46      39      39
P4      4       5       2               18      14      9       1
P5      6       3       1               9       3       0       0
P6      7       6       3               41      34      28      28
P7      9       4       2               22      13      9       9
P8      11      5       1               16      5       0       0
P9      13      3       4               51      38      35      35
P10     15      4       2               26      11      7       7

Average Waiting Time = 15.20
Average Turnaround Time = 20.30
```

**Conclusion:**

The experiment conducted involved the implementation of a C++ program to demonstrate the Priority Scheduling algorithm (non-pre-emptive and same arrival time of process), resulting in an improved understanding of scheduling algorithms and their implementation. The experiment allowed for the demonstration of the Priority algorithm in scheduling processes and calculating waiting time, turnaround time, and completion time. It helped to understand the working mechanism of a non-preemptive Priority Scheduling Algorithm.

**Assessment Schemes:**

| Attendance | Discipline | Short oral | Correctness of Lab Report | Timely completion of Lab Report | Total marks (10) | Signature of Teacher with Date |
|---|---|---|---|---|---|---|
| | | | | | | |